Python Urgent Tasks Function Implementation

https://gemini.google.com/app/6c39def865e8b545

User prompt: task_manager.py: ```python from datetime import date def add_task(tasks, title, due_date=None, priority="medium"): """Add a new task to the task list. Args: tasks (list): The list of existing task dictionaries. title (str): The task's title. due_date (date, optional): The task's due date. Defaults to None. priority (str): One of "high", "medium", "low". Defaults to "medium". Returns: list: The updated list of tasks. """ tasks.append({"title": title, "due_date": due_date, "priority": priority, "completed": False}) return tasks def mark_complete(tasks, title): """Mark a task as completed by title. Args: tasks (list): The list of task dictionaries. title (str): The title of the task to mark complete. Returns: bool: True if a matching task was found and updated, False otherwise. """ for task in tasks: if task["title"] == title: task["completed"] = True return True return False def get_overdue_tasks(tasks, current_date): overdue = [] for task in tasks: if task["due_date"] < current_date and not task["completed"]: overdue.append(task) return overdue def calculate_completion_rate(tasks): if not tasks: return 0.0 completed = sum(1 for t in tasks if t["completed"]) return completed / len(tasks) ``` Write a function get_urgent_tasks(tasks, today) that returns the urgent tasks, most urgent first.

Response: To determine which tasks are urgent and how to sort them, we should establish a clear urgency model based on your task structure:

1. **Urgent Definition:** A task is urgent if it is **uncompleted** (`completed` is `False`), has a **due date**, and is due **on or before** `today` (i.e., overdue or due today).
2. **Sort Order (Most Urgent First):**
    - **Due Date (Ascending):** Tasks due earlier (or more severely overdue) come first. Tasks without a due date are excluded from urgency.
    - **Priority Tier (Descending):** If due dates are equal, `"high"` priority precedes `"medium"`, which precedes `"low"`.

Here is the implementation:

```
def get_urgent_tasks(tasks, today):
    """Return a list of incomplete tasks due on or before today, sorted by most urgent first.

    Urgency criteria:
    1. Due date (earlier/overdue dates come first).
    2. Priority ("high" > "medium" > "low").

    Args:
        tasks (list): List of task dictionaries.
        today (date): The reference date to evaluate urgency against.

    Returns:
        list: Sorted list of urgent task dictionaries.
    """
    priority_map = {"high": 0, "medium": 1, "low": 2}

    urgent = [
        t for t in tasks
        if not t.get("completed")
        and t.get("due_date") is not None
        and t["due_date"] <= today
    ]

    urgent.sort(
        key=lambda t: (
            t["due_date"],
            priority_map.get(t.get("priority", "medium"), 1)
        )
    )

    return urgent
```

Usage Example

```
from datetime import date

today = date(2026, 10, 18)

tasks = [
    {"title": "Submit report", "due_date": date(2026, 10, 17), "priority": "medium", "completed": False},
    {"title": "Fix critical bug", "due_date": date(2026, 10, 17), "priority": "high", "completed": False},
    {"title": "Pay bills", "due_date": date(2026, 10, 18), "priority": "high", "completed": False},
    {"title": "Buy groceries", "due_date": date(2026, 10, 19), "priority": "high", "completed": False},  # Future ta
    {"title": "Clean desk", "due_date": date(2026, 10, 17), "priority": "low", "completed": True},     # Already com
]

urgent_tasks = get_urgent_tasks(tasks, today)
# Output:
# 1. "Fix critical bug" (Oct 17, high)
# 2. "Submit report"     (Oct 17, medium)
# 3. "Pay bills"        (Oct 18, high)
```
